

Topic 7

Advanced SAT Techniques

系統晶片驗證
SoC Verification

2025.04.09

What affect the SAT efficiency?

1. Decision order
2. Logic implication (Boolean Constraint Propagation, BCP)
3. Various learning techniques
4. Database simplification

Impact of Decision Ordering

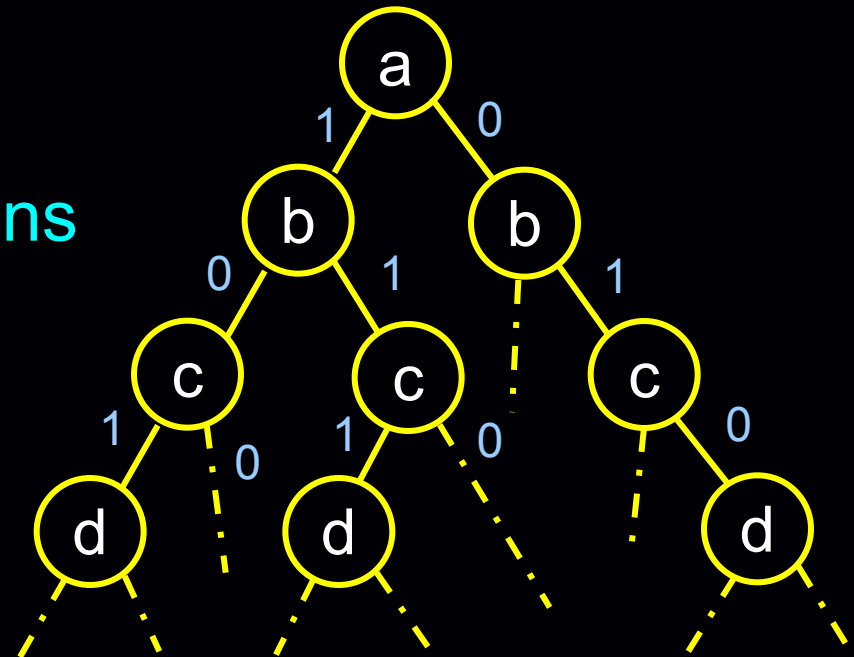
- ◆ Decision ordering: the order of gates that the corresponding decisions are made

1. Order of gates

2. Decision values

→ Good and bad decisions can lead to exponential difference

(e.g. 2^{10} vs. 2^{50})



- ◆ (Think) Does the decision value matter? (i.e. should we decide on '1' or '0' first?)

Static Decision Ordering

- ◆ Decision order and values are pre-computed in the beginning and remain unchanged
- 1. Topological
 - Depth-first
 - Breadth-first
 - Guided by gate types
- 2. Probability-based
 - Controllability / Observability
 - Signal probability
 - (Weighted) Random
- 3. Influence-based
 - Literal count
 - #fanins / #fanouts
 - Influence of implications

Dynamic Decision Ordering

- ◆ Decision order and values are dynamically determined based on current implication values, justification frontier, etc.
 - Use similar criteria as static method
 - But can mix different rules dynamically
 - ◆ Pros
 - May lead to better decisions
 - Avoid useless decisions
 - ◆ Cons
 - Overhead in computing dynamic ordering may be high
 - Effectiveness sometimes is hard to predict
- However, experiences show that the best is:
1. Has a good initial decision ordering
 2. Adaptively adjust the decision order after a certain amount of backtracks

zChaff's Variable State Independent Decaying Sum (VSIDS) Decision Heuristic

- (1) Each variable in each polarity has a counter, initialized to 0.
- (2) When a clause is added to the database, the counter associated with each literal in the clause is incremented.
- (3) The (unassigned) variable and polarity with the highest counter is chosen at each decision.
- (4) Ties are broken randomly by default, although this is configurable
- (5) *Periodically, all the counters are divided by a constant.*

Berkmin – Decision Making Heuristics

E. Goldberg, and Y. Novikov, “BerkMin: A Fast and Robust Sat-Solver”,
Proc. DATE 2002, pp. 142-149.

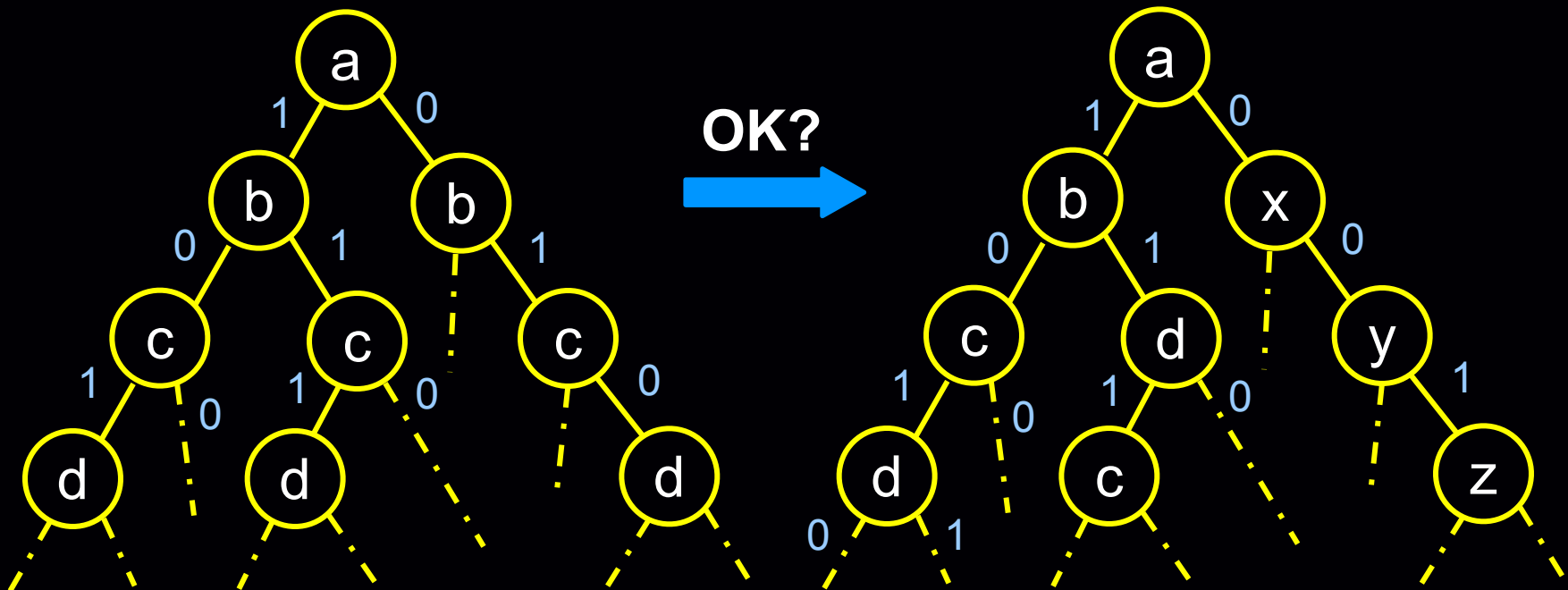
- ◆ Identify the most recently learned clause which is unsatisfied
- ◆ Pick most active variable in this clause to branch on
- ◆ Variable activities
 - updated during conflict analysis
 - decay periodically
- ◆ If all learnt conflict clauses are satisfied, choose variable using a global heuristic
- ◆ Increased emphasis on “locality” of decisions

More decision heuristics...

- ◆ Variable Move-To-Front (VMTF)
 - ◆ Clause Based Heuristic (CBH)
 - ◆ Resolution Based Scoring (RBS)
 - ◆ ...
-
- ◆ In general, there is no single decision heuristic that works for every case.
 - How to adaptively move to a good decision heuristic may be the winner...

A closer look at binary decision tree

Should the decision orderings on all branches be the same?



Remember when we talked about
conflict-driven learning,
we mentioned that

by adding learned clauses

we can do non-chronological backtracking, while
still achieve complete proof,

and the decision tree becomes a decision stack!

How??

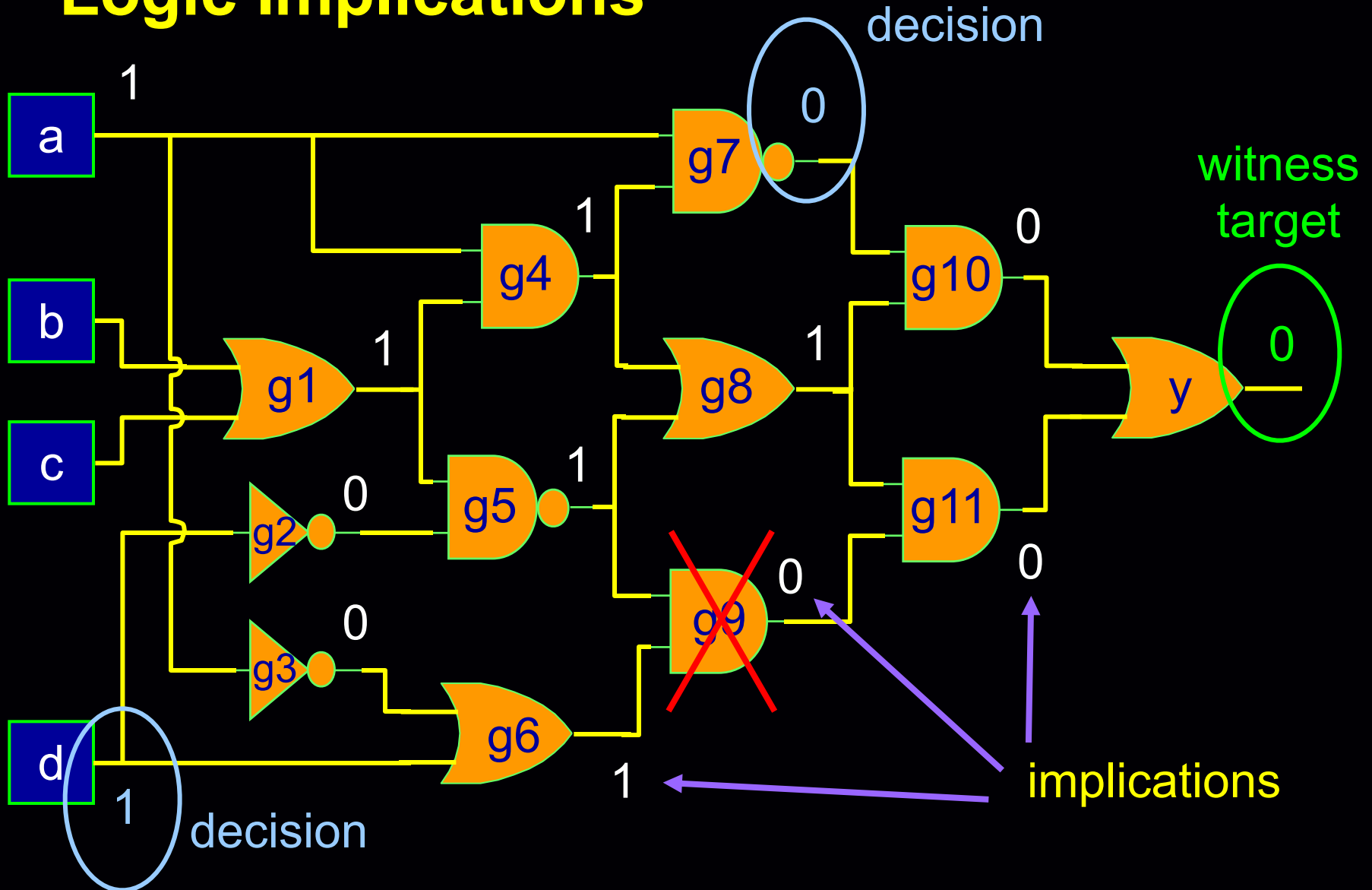
The Constraint Refinement Process

- ◆ Search region is gradually narrowed down by the learned constraints
- ◆ Learned information is universally true
 - Independent of the target implication, only related to the circuit function
 - The proof efforts between different properties can be shared
 - Incremental SAT
- ◆ Decision process can “restart” any time any where!!
 - Can use different decision ordering to explore different area in the decision tree
 - Previous efforts will not be wasted

What affect the SAT efficiency?

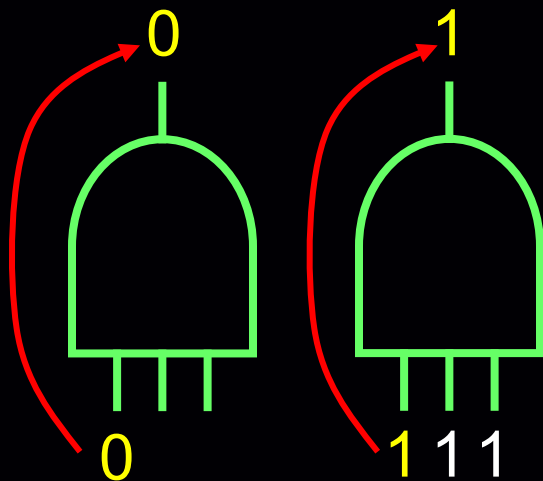
1. Decision order
- ▶ 2. Logic implication (Boolean Constraint Propagation, BCP)
3. Various learning techniques
4. Database simplification

Logic Implications

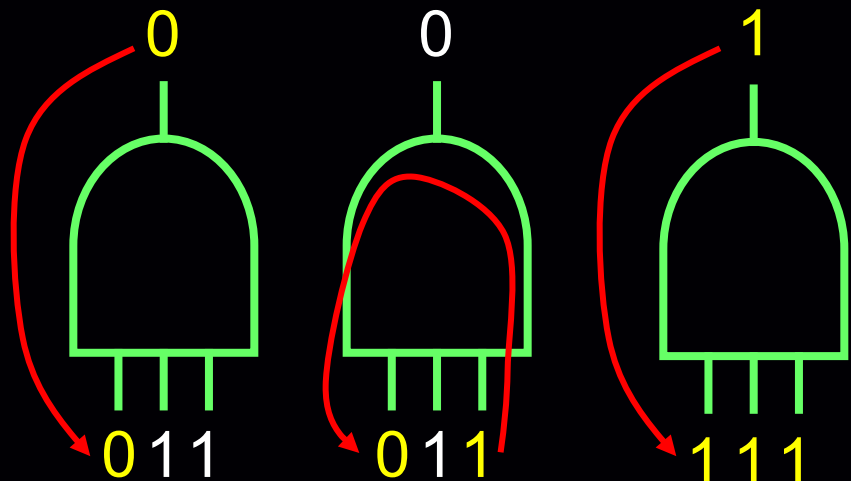


Logic Implications

- ◆ Also called “Boolean Constraint Propagation” (BCP)
- ◆ Take over 90% of the SAT runtime
- ◆ Imply values to other gates in both forward and backward directions

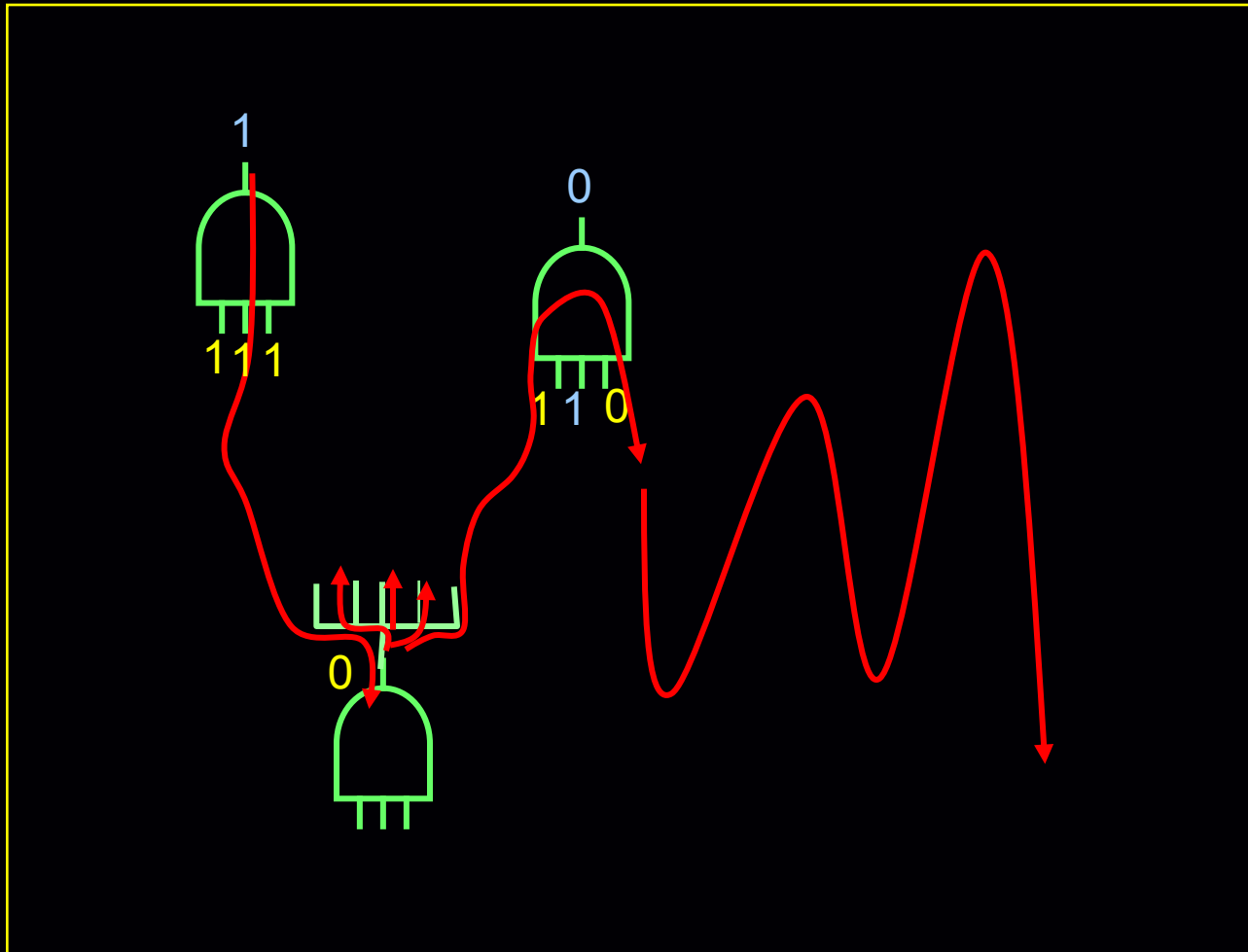


Forward implications



Backward implications

“Omni-directional” in a netlist

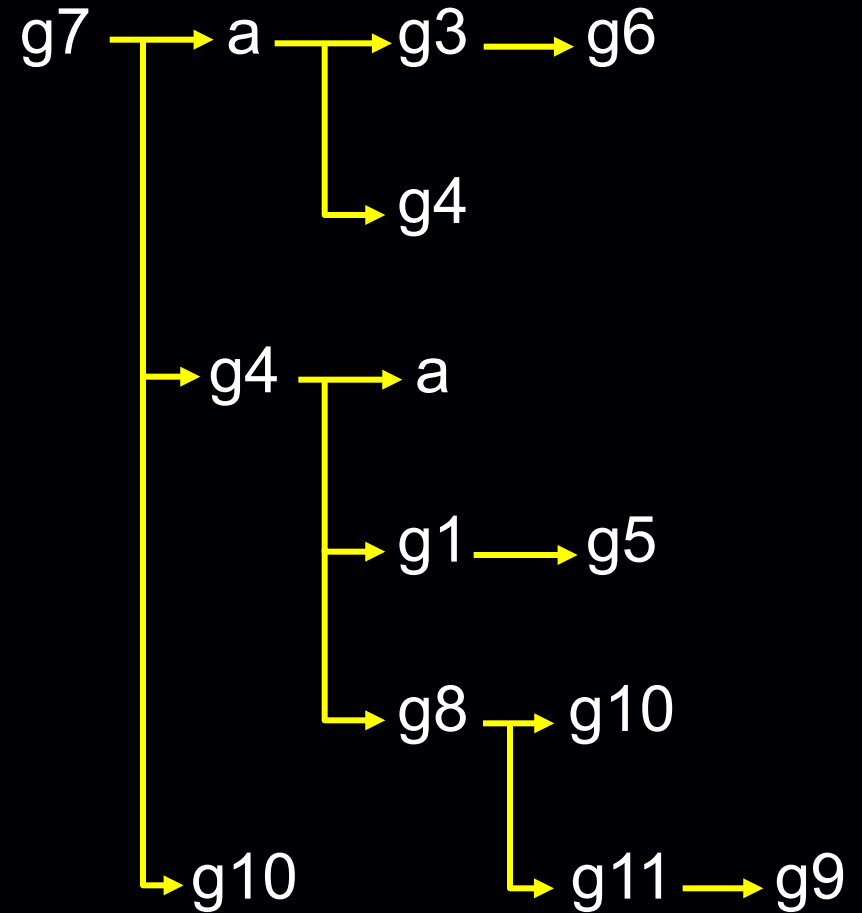
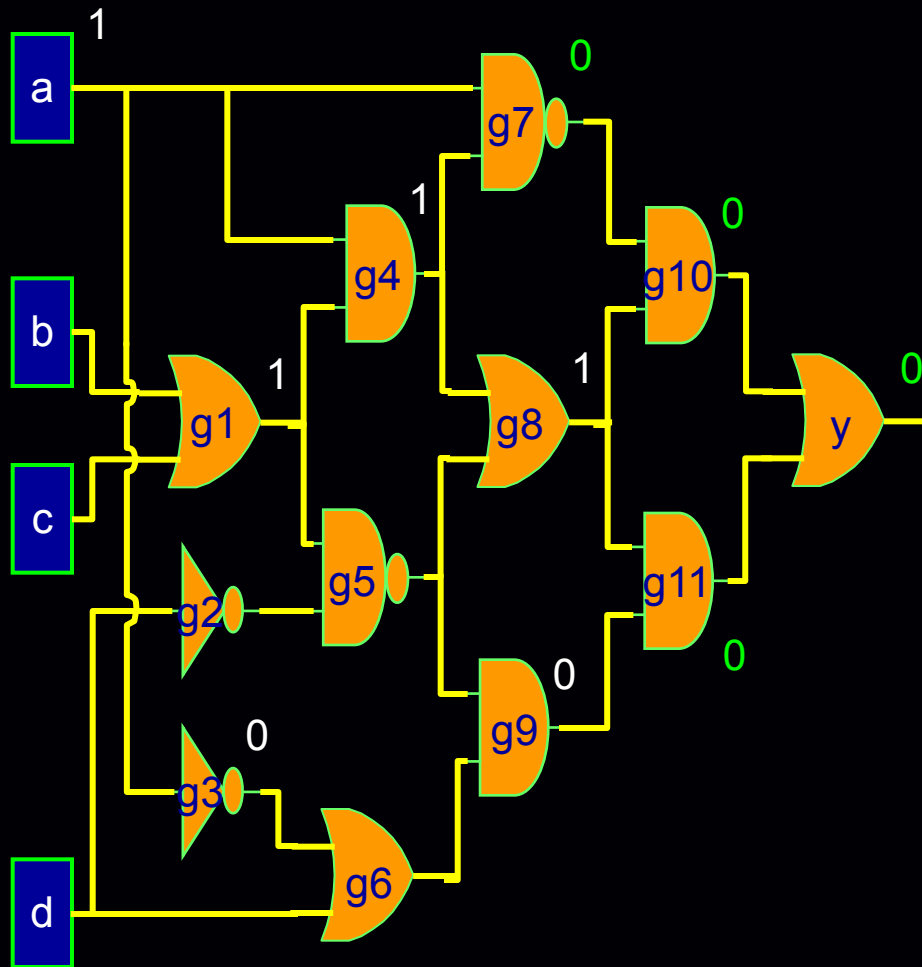


How to **schedule** and **evaluate**
these implications??

Take 1: Logic Implication by Recursion

```
1. bool logicImplication(Gate* g, value v)
2. {
3.     if (g->hasConflictValue(v)) return false;
4.     if (g->getValue() != 'x') return true;
5.     // g has value 'x' → can be implied
6.     g->setValue(v);
7.     // fanins and fanouts that may be
8.     // implied by g
9.     List scheduleList = checkImplication(g);
10.    for_each_gate_value(scheduleList, gg, vv)
11.        if (logicImplication(gg, vv) == false)
12.            return false;
13.    return true;
14. }
```

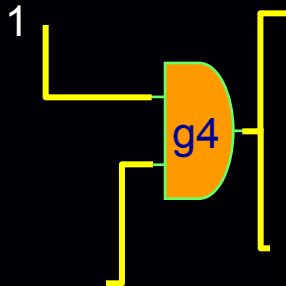
Logic Implication by Recursion Example



Recursion tree

Logic Implication by Recursion

- ◆ Pros
 - Easy to implement
- ◆ Cons
 - A gate may be put into **scheduleList** many times by different neighboring gates
 1. Value has already been implied, or
 2. Too early to evaluate



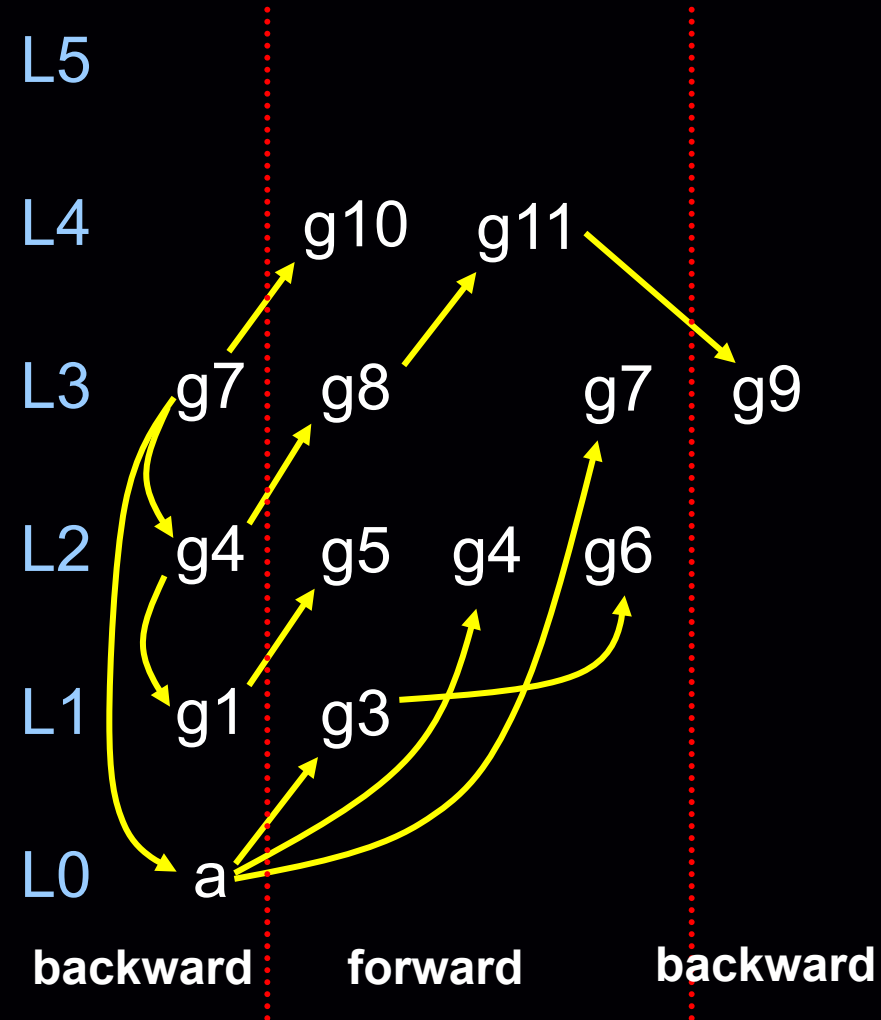
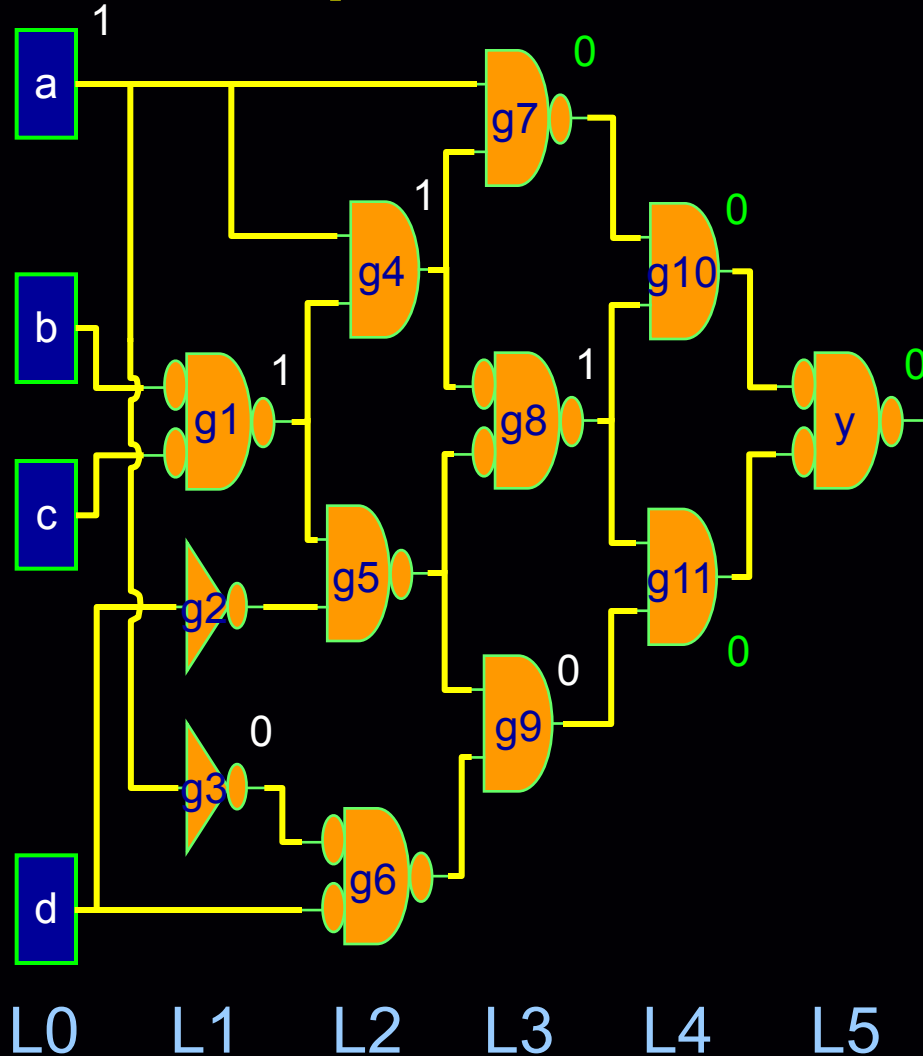
Take 2:

Logic Implication by Topological Order

- ◆ Preprocess
 - All gates are sorted topologically
 - Each gate has a field “_level” = $\max(\text{fanin} \rightarrow \text{_level}) + 1$
- ```
1. bool logicImplication()
2. {
3. while (more scheduled gates)
4. for scheduled gates (level = maxLevel to 0)
5. apply backward implications and
6. schedule forward implications
7. return false if any conflict;
8. for scheduled gates (level = 0 to maxLevel)
9. apply forward implications and
10. schedule backward implications;
11. return false if any conflict;
12. return true;
13. }
```

# Logic Implication in Topological Order

## Example



# Logic Implication in Topological Order

## ◆ Pros

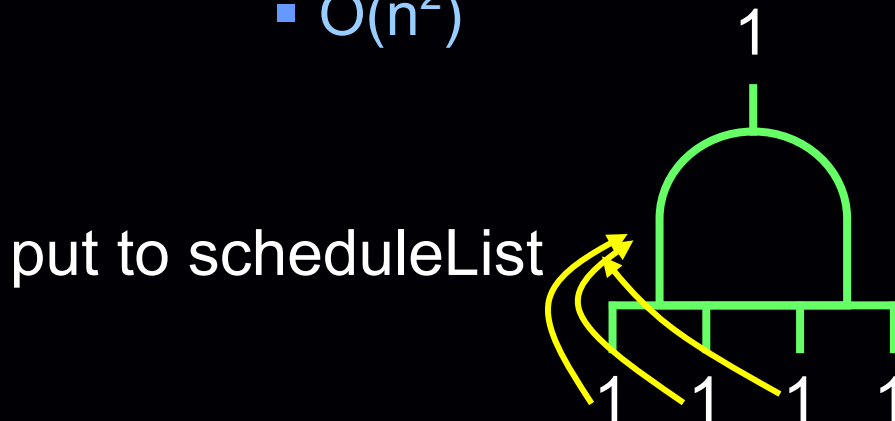
- Minimize the number of scheduling
- Each gate is checked only once in each forward or backward implication routine

## ◆ Cons

- A gate may still be checked more times than necessary
  1. Value has already been implied, or
  2. Too early to evaluate

# Fruitless Implication Checking

- ◆ Checking if a gate can be implied, but usually no implication
- ◆ For example, consider the all-1's forward implication of an AND gate
  - Only the last '1' can trigger the forward implication
    - The first (n-1) checks are useless
  - Worse case: for n-input AND gate
    - Need to check  $(1 + 2 + \dots + n)$  times of fanins
    - $O(n^2)$



# Trying to avoid fruitless implication checking

- ◆ Use a **counter** to record how many 'x' are in the fanins of a gate (e.g. all 1's for an AND gate)
  - Decrement by 1 when fanin is implied
  - Increment by 1 when fanin value is backtracked
  - When no 'x' fanin (x count = 0) → forward implication
- ◆ Although this can avoid fruitless implication checking, but the overhead in maintaining the counts could be an overkill...
  - 'x' count could be:  $5 \rightarrow 4 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 4 \rightarrow 5 \dots$



Can we do better?

Why should we check the pins  
when there are more than 1 x's?

But, how do we know if there  
are more than 1 x's?

# Logic Implication for CNF-Based SAT

## ◆ Refresh:

- Boolean function is represented as a CNF (i.e. Product of Sum, POS format)
  - Can arbitrary Boolean function be converted into CNF?
- e.g.  $(a+b+c)(a'+b'+c)(a'+b+c')(a+b'+c')$
- To be satisfied, all the clauses should be '1'

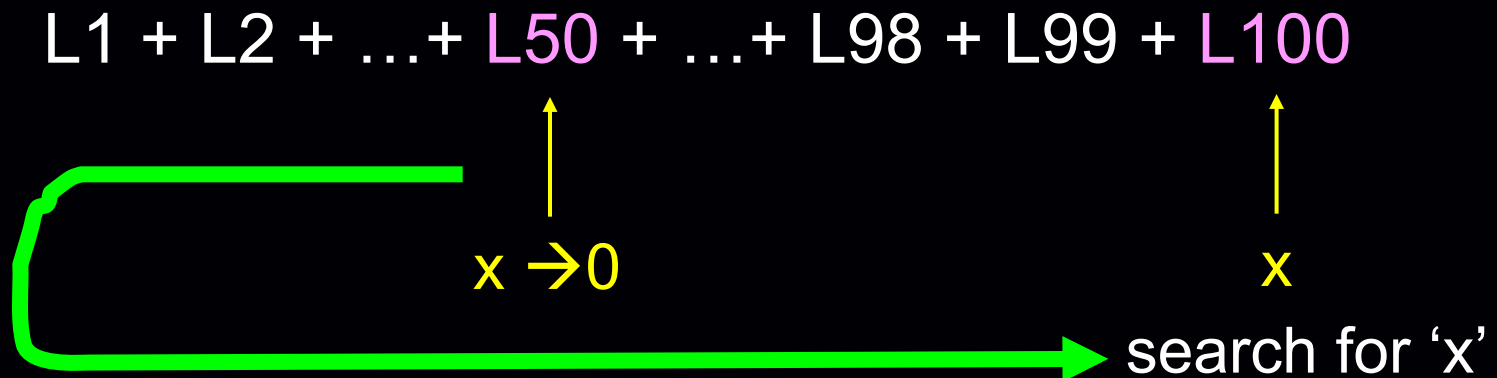
## Logic implication for CNF-based SAT is simple ---

- ◆ If a literal in a clause gets an implication '1'
  - The clause is satisfied
- ◆ If a literal in a clause gets an implication '0'
  - Check: how many literals in the clause have unknown value?
  - $\geq 2$  : no operation
  - $= 1$  : the remaining literal will be implied '1' (unit clause rule)
  - $= 0$  : the clause is evaluated to '0' → a conflict !!

# 2-Watched-Literal Algorithm

H. Zhang, SATO, CADE 97; M. Moskewicz *et al*, Chaff, DAC 2001

- ◆ For each clause, keep 2 pointers on 2 literals that have “non-0” values
  - If any watched literal gets implication ‘0’
    - Scan in the clause for another literal with “non-0” value
    - If found, update the watched literal pointer
    - Else, if the other watched literal is '0' => return Conflict
      - Else, imply the other watched literal with value ‘1’



# Watching Clauses of a Literal

$$L1 + L2 + \dots + L50 + \dots + L98 + L99 + L100$$

$\uparrow$                        $\uparrow$                        $\uparrow$   
 $x$                        $x \rightarrow 0$                        $x$

- ◆ Let L50 and L100 be the watched literals of the clause
- ◆ What happens if L98 gets an implication  $x \rightarrow 0$ ? Update watch?
  - No action
  - But how does this clause know that it doesn't have to update the watched literals?
  - Check if L98 is a watched literal?

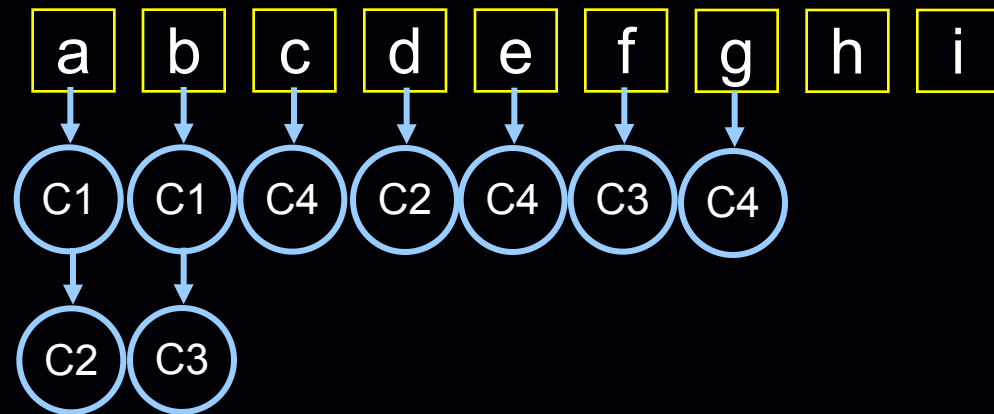
**No!!** By keeping a list of "watching clauses" for each literal (who is watching me), when a literal is implied, it only has to update the clauses that watch this literal

## 2-Watched-Literal Algorithm Example

Each clause stores:  
2 watched literal pointers

C1: (a + b + c + d)  
C2: (a + d + e + f + g)  
C3: (b + f)  
C4: (c + e + g + h + i)

Each literal stores:  
A list of watching clauses



$c \leftarrow 0$

- Update watched literal pointer for C4 (for example, to 'g')
- Erase c's watching-clause list
- Add 'C4' to g's watching-clause list

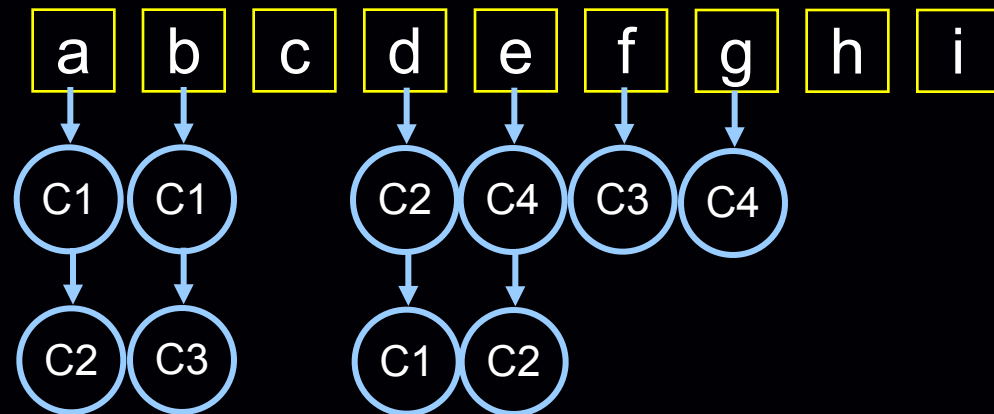
[Note] Don't need to check 'C1'

## 2-Watched-Literal Algorithm Example

Each clause stores:  
2 watched literal pointers

C1: (a + b + c + d)  
C2: (a + d + e + f + g)  
C3: (b + f)  
C4: (c + e + g + h + i)

Each literal stores:  
A list of watching clauses



$a \leftarrow 0$

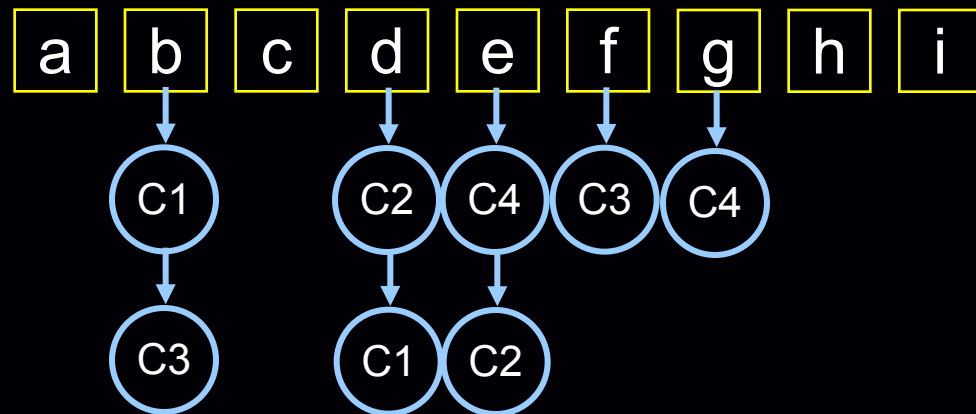
- Update watched literal pointer for C1 (only choice, to 'd')
- Update watched literal pointer for C2 (for example, to 'e')
- Erase a's watching-clause list
- Add 'C1' to d's and 'C2' to e's watching-clause lists

# 2-Watched-Literal Algorithm Example

Each clause stores:  
2 watched literal pointers

C1: ( a + **b** + c + **d** )  
C2: ( a + **d** + **e** + f + g )  
C3: ( **b** + **f** )  
C4: ( c + **e** + **g** + h + i )

Each literal stores:  
A list of watching clauses



$b \leftarrow 0$

- No more unknown literal for C1 :  $d = 1$
- No more unknown literal for C3 :  $f = 1$

[Note] No change on watched literals

## zChaff's improvements on the 2-watch scheme

- ◆ In the original 2-watch scheme paper (SATO[97]), watched literals are restrained to head and tail of the non-zero literals
  - Updates of the watched literals can only move in specific directions

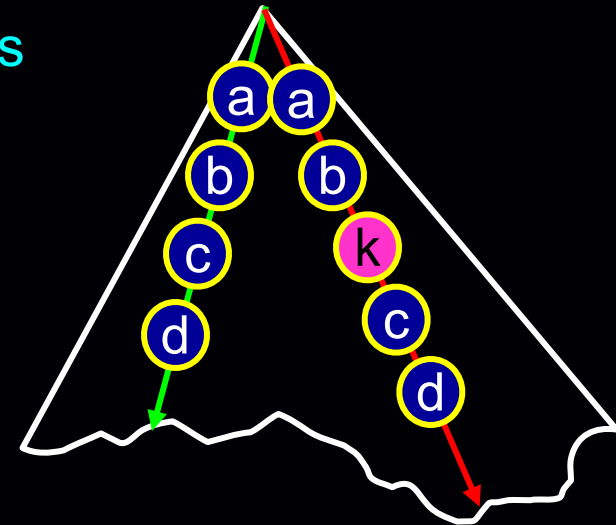


- When backtracking, need to reset the watched literals to the previous positions
- ◆ In zChaff, updates of the watched literals can move in any direction
  - Usually follow a single (forward) direction so that the watched literals will move to the "last implied positions"
  - No need to reset the watched literals when backtracking
  - What's the advantage?



# Caching Effect: Reducing from $O(n)$ to amortized $O(C)$

- ◆ The fact
  - Most of the time, the decision orderings at different parts of the decision tree are quite similar during a proof (or even from proof to proof)
    - Literals in a clause get the implications almost **by the same order** every time
- ◆ Watched literals
  - point to the last implied literals
  - Don't update watched literals after backtrack.
  - After backtracks, very likely no updates will be evoked for the clauses of the other unwatched literals.



$$(L1 + L2 + L3 + L4 + L5 + L6)$$

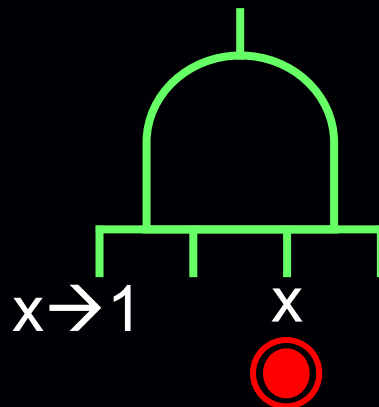


Logic implication can be very efficient for CNF-based SAT by using “watch” scheme.

Can this idea be applied to circuit-based SAT?

# Watched-fanin concept

- ◆ (Trial) In the n-input AND gate case, keep a pointer to one of its fanin that has value 'x' (watched fanin)
  - If other fanin gets implication '1' → no operation
  - If this watched fanin gets implication '1', try to find another 'x' fanin to be **new watched fanin**
    - If found, update the pointer
    - If not found → imply '1' on the gate output



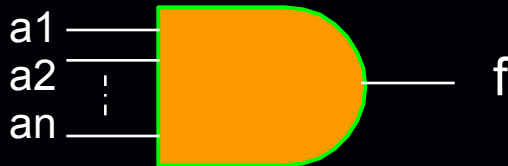
Sounds good for all-1's forward implication of an AND gate, but what about 0 implication, backward implication, and other types of gates?

**Different watched schemes?**  
(Could be very complex...)

That's one of the reasons why simpler data structure like **CNF-based SAT** engine can be more efficient sometimes

# Difference between circuit and CNF SAT

## ◆ Circuit-based SAT: gates

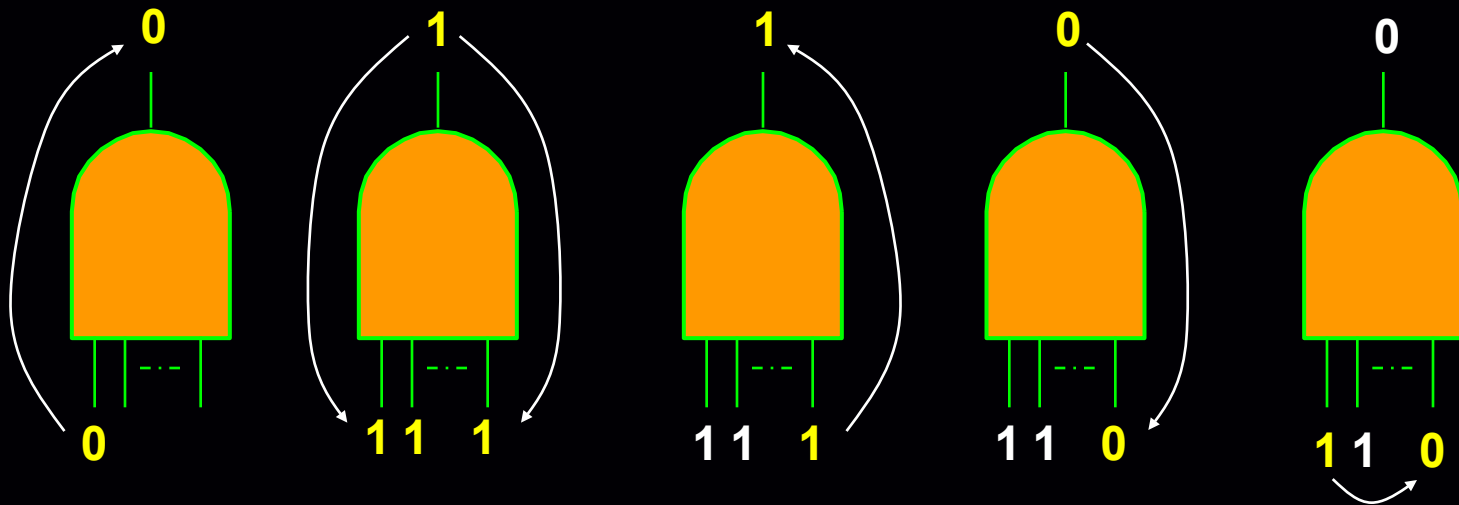


## ◆ CNF-based SAT: clauses, variables, literals

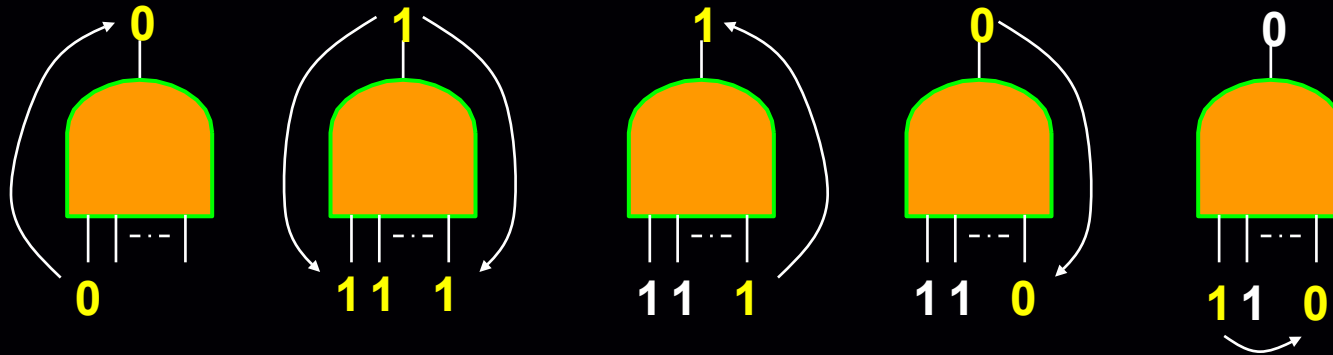
- $n+1$  clauses
  - $(a_1 + f')(a_2 + f') \dots (a_n + f')$
  - $(a_1' + a_2' + \dots + a_n' + f)$
- 1 variable ( $f$ )
- 2 literals ( $f, f'$ )

# Any problem?

- ◆ Remember there is only one type of implication in CNF SAT
  - Logic implication can be very efficient
- ◆ But implications on an AND gate are trickier



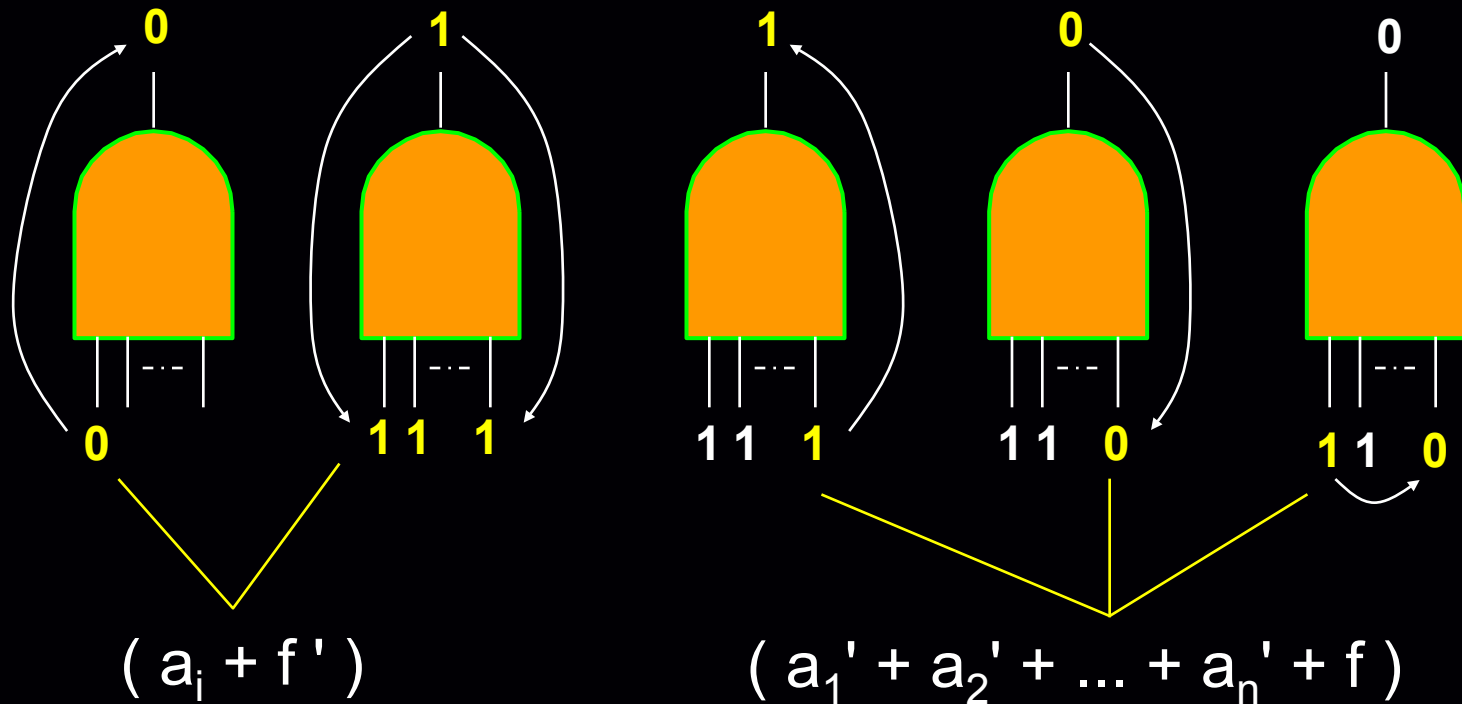
# Can circuit SAT do 2-watch?



## ◆ Watch 2 fanins?

- What's the watch value?
- How about gate output?
- How about OR, NOR, NAND,.. gates?
- How about XOR, MUX, ... complex gates?

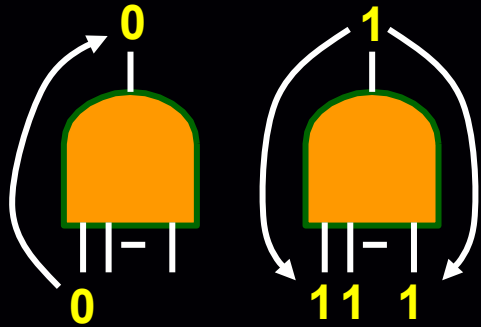
# A Closer Look



**Different implications on circuit-based SAT  
actually map to the same clause on CNF SAT**



# Direct vs. Indirect Implications



## 1. Direct implication

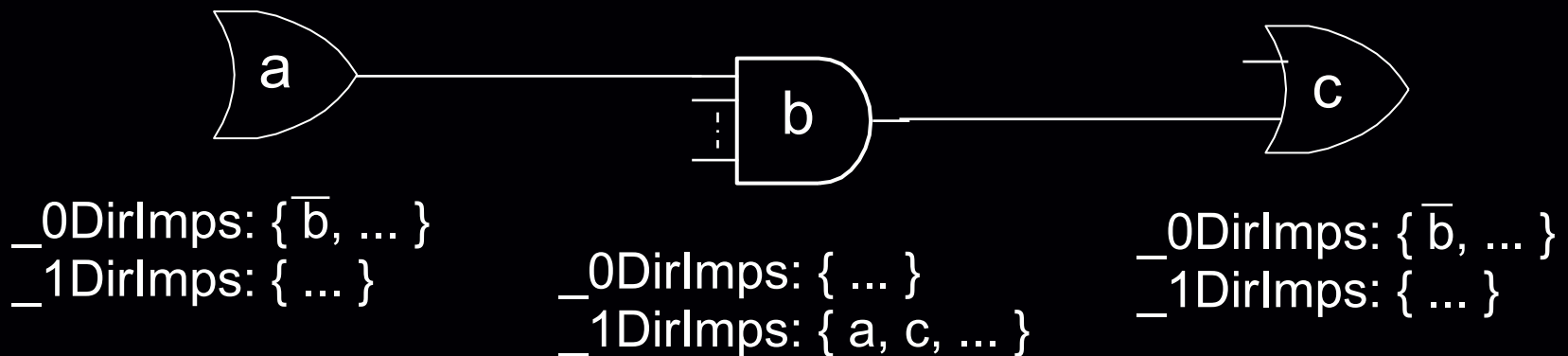
- Corresponding to  $n$  2-literal clauses in CNF SAT
  - Single implication source
- No need to watch

# Direct Implication

1. Single source for each implication
2. **Only depends on netlist structure**; has nothing to do with the proving process (e.g. decisions, etc)
3. Should never encounter “CONFLICT” during the proof process (assume circuit is a DAG)

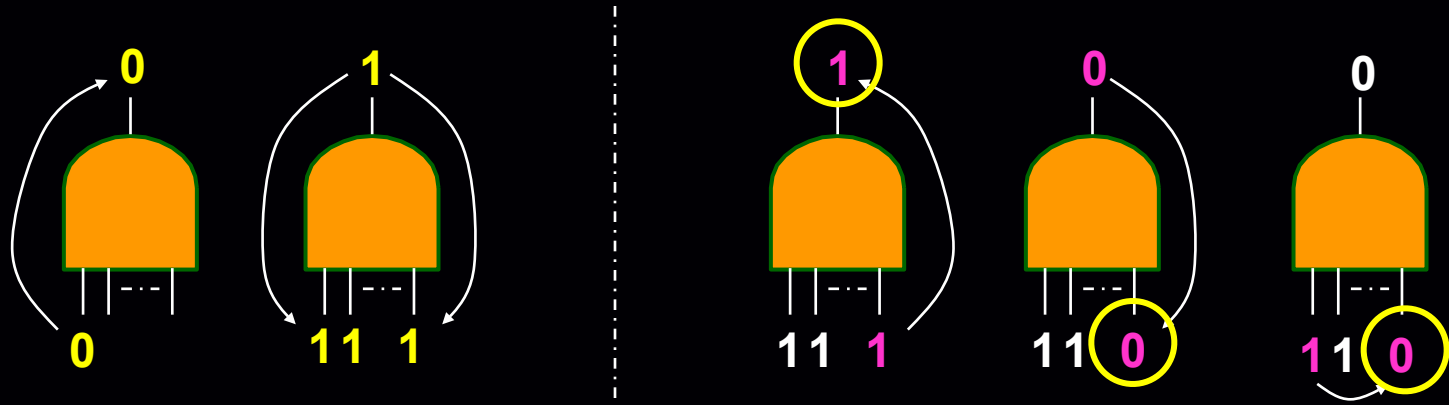
# Direct Implication

- Construct a “direct implication graph” in the preprocessing step
- Apply direct implications whenever a gate is implied to a value



- For example, when **b** is implied '1', directly iterate through its  $\_1\text{DirImps}$  and imply **a = 1** and **c = 1**

# Direct vs. Indirect Implications



## 1. Direct implication

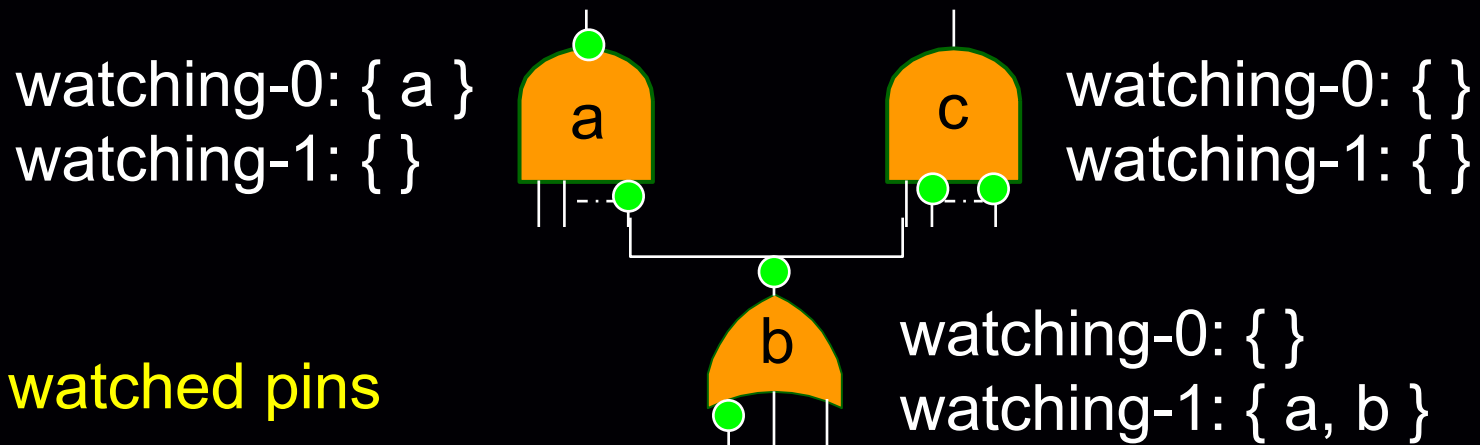
- Corresponding  $n$  2-literal clauses in CNF SAT
  - Single implication source
- No need to watch

## 2. Indirect implication

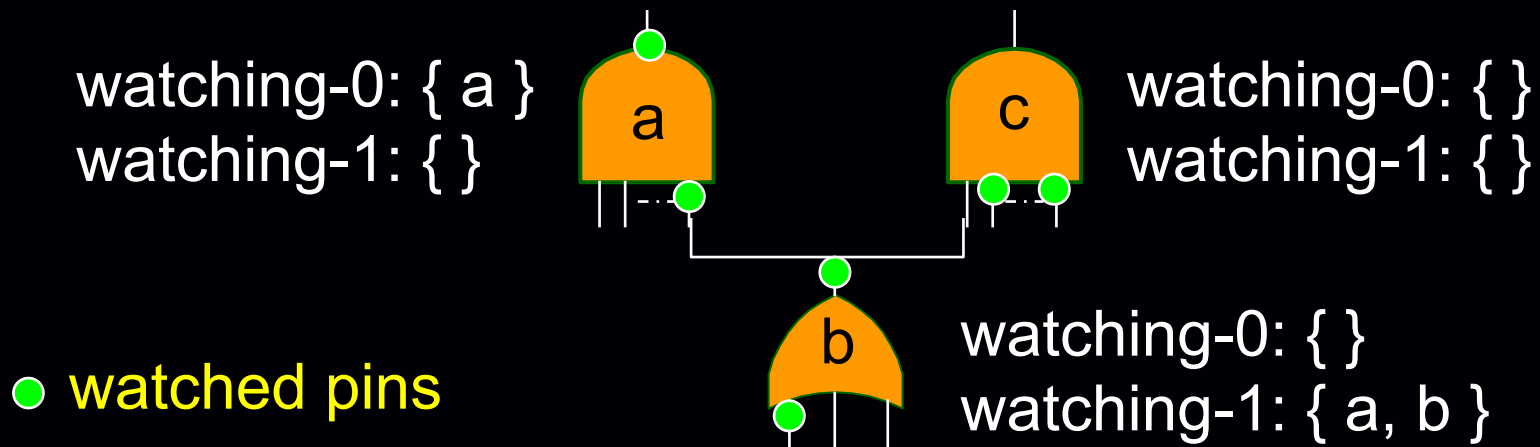
- Corresponding to the same  $(n+1)$ -literal clause
  - Only the last implied pin has different value
- 2 watches: among all fanins and the gate itself

# Indirect Implication (AND/OR gate)

- ◆ Select 2 pins (fanins or the gate itself) in a gate to watch
  - Almost the same as CNF SAT, except that the “watched value” depends on the gate type and I/O
    - For AND (OR) gate, watch 1 (0) for inputs, and 0 (1) for output
    - Watched pins must have non-watched values
  - For each gate, two lists of watching gates
    - When a gate gets a value, perform direct implication and then update watch for the gates on the watching list



# Indirect Implication (AND/OR gate)

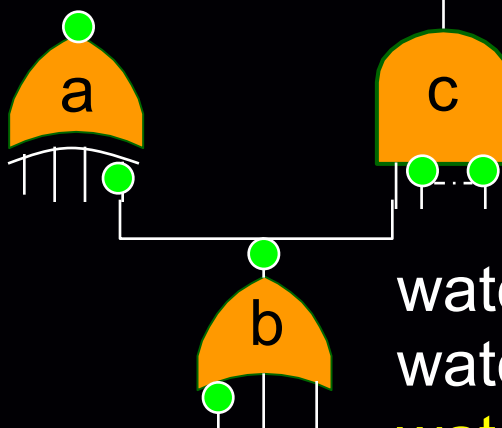


- ◆ 'a' watches on itself (output, watch-0) and one of its fanins (watch-1)
  - 'a' is in the watching-0 list of itself, and is in the watching-1 list of 'b'
- ◆ If 'b' is implied '1' --
  - Pick 'a' and 'b' from its watching-1 list to update the watched pins
    - 'a' has to update the watched pin for its fanning (to another non-1 fanin)
    - 'b' has to update the watched pin for its fanning (to another non-0 fanin)
    - No action needed for 'c' (because the corresponding fanin is not watched)

# Watch Scheme for XOR Gate (How?)

- ◆ n-input XOR gate
  - $2^n (n+1)$ -literal clauses
  - e.g.  $(a + b + f) (\bar{a} + b + f) (a + \bar{b} + f) (\bar{a} + \bar{b} + f)$
- ◆ Implication occurs only when n variables become “known”
  - 2-watch; watch-known

watching-0: { }  
watching-1: { }  
watching-known: { a }



watching-0: { }  
watching-1: { }  
watching-known: { }

watching-0: { }  
watching-1: { b }  
watching-known: { a }

# Generic Watch Scheme

- ◆ It can be shown that the watch scheme can be extended to complex gates such as MUXes, Pseudo Boolean gates, etc.
- ◆ For more details, please refer to:
  - "QuteSAT: A Robust Circuit-based SAT Solver for Complex Circuit Structure", DATE 2007.



# Various Learning Techniques

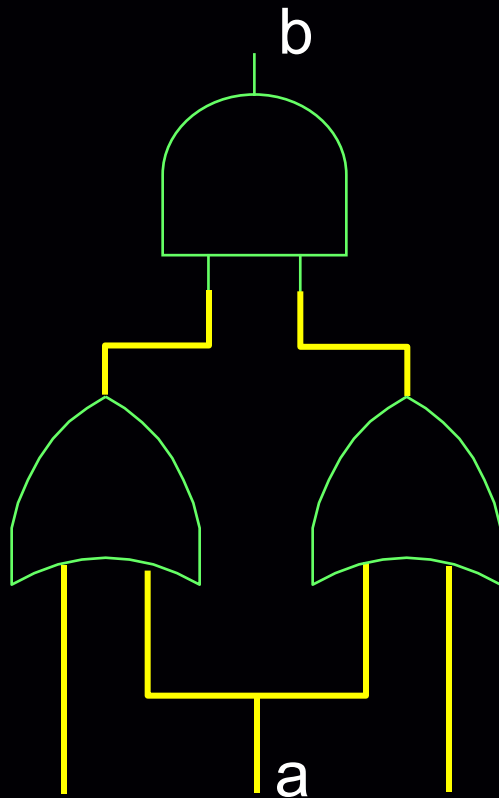
- ◆ Other than conflict-driven learning, there are many other learning techniques that can help
  - Derive more implications  
→ may help find the conflict earlier
  - Provide information for decision ordering
- 1. Static learning
- 2. By signal correlations
- 3. Recursive learning
- 4. Success-driven learning

# Static Learning

- ◆ Learn by contraposition

$$(a \rightarrow b \equiv !b \rightarrow !a)$$

- ◆ e.g.



For the multi-source implications:

$$a = 1 \rightarrow b = 1$$

$$\text{Learned } b = 0 \rightarrow a = 0$$

The question is:  
which gate to learn??

Ref: "SOCRATES: A Highly Efficient Automatic Test Pattern Generation System", Schulz *et.al*, TCAD 1988

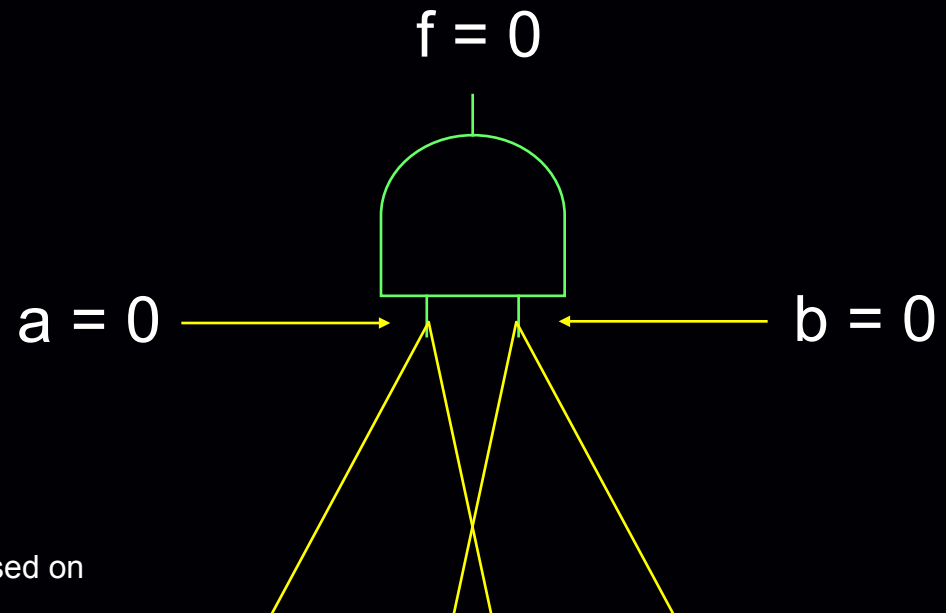
# Learned by Signal Correlations

- ◆ A proof-based approach
  - Since learned information is universally true, we can create some **internal interesting properties**, and use these properties to derive some interesting learning (by conflict analysis)
- ◆ e.g. By simulation, if we find a gate 'g' is very likely to stuck at some value 'v'
  - Witness " $g = \sim v$ " (should produce many conflicts)
- ◆ e.g. By simulation, if two signals respond almost the same
  - Witness " $p \neq q$ "
- ◆ No matter the proof is finished or not
  - We can always learn something

Ref: Feng Lu, *et. al*, "A Circuit SAT Solver with Signal Correlation Guided Learning", DATE 2003

# Recursive Learning

- ◆ To justify  $f = 0$ 
  - $(a = 0) \text{ or } (b = 0)$
  - Let  $S_a$  and  $S_b$  be the set of implications from  $(a = 0)$  and  $(b = 0)$ , respectively
  - Let  $S = S_a \cap S_b$ 
    - $(f = 0)$  implies  $S$
- ◆ A recursive process
- ◆ Deep recursion could be very expensive
- ◆ The problem is:  
Difficult to record the learned information



Ref: "HANNIBAL: an efficient tool for logic verification based on recursive learning", Wolfgang Kunz, ICCAD 1993

# Conflict vs. Success-Driven Learning

**Motivation:** Traditional SAT approach finds only 1 solution, can we find more (or all) the solutions?

- ◆ How to record the solutions?
  - Hash table? (too expensive)
- ◆ Success-driven learning
  - Similar to conflict learning
  - When we find one solution, say  $(v_1, v_2, \dots, v_n)$ , add a blocking gate “ $v_1 \ \&\& \ v_2 \ \&\& \ \dots \ v_n = 0$ ” so that
    - This solution won't be repeated
    - May lead to new implication
    - Can continue the justification process for the next solution
  - At the end, all the solutions are recorded as set of blocking gates (or clauses)

# Conflict vs. Success-Driven Learning

- ◆ However, the number of solutions in a SAT problem can be very huge!!

→ Some solutions may look alike ---

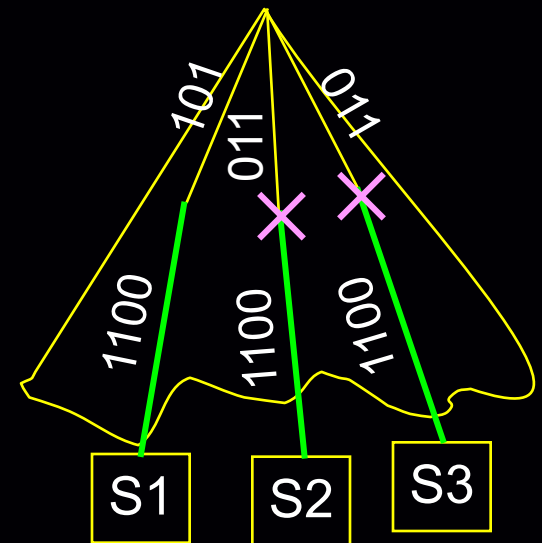
e.g. 1010011, 1100011, 0110011...

s1

s2

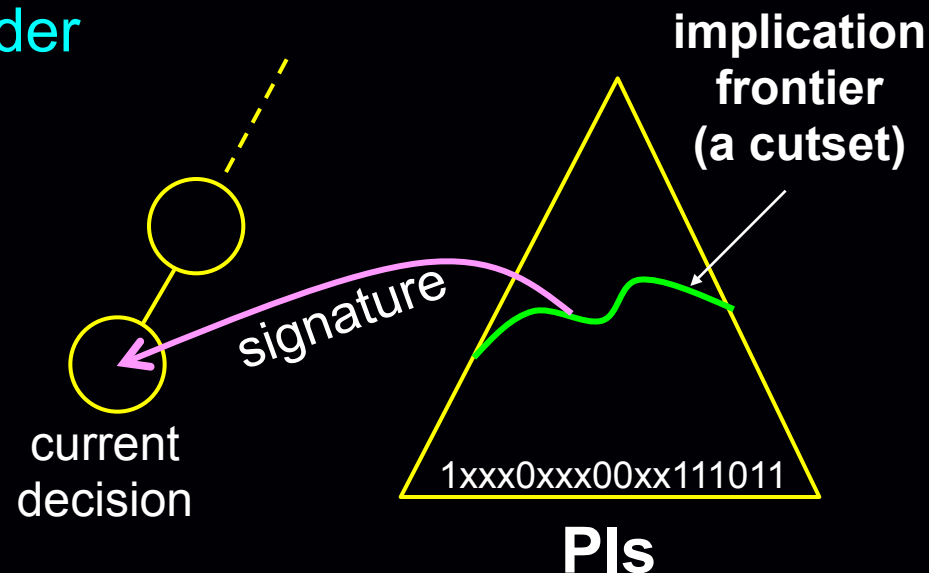
s3

**Can we predict that the sub-solutions under the sub-search tree are already covered?**



# Success-Driven Learning

- ◆ Ref: Shuo, et al. DATE 2003
- ◆ Assume
  - ATPG-based technique (work on circuit)
  - Decisions on PIs only → forward implications
- ◆ Search State Equivalence
  - If two decisions have the same signature
    - The “sub-solutions” under the sub-search space are the same!!
    - No need to search
- ◆ Note: they also store the solutions in a “free BDD”



Although “learning” in general can lead to more implications and possibly lead to conflicts earlier (i.e. bound earlier) ---

1. It may slow down the implication process
2. It may affect the decision ordering, which may not necessarily reduce the #decisions



# What can we do to make the learning useful?

1. Use learning to find better decision ordering
  - zChaff uses learned information to refine the decision ordering
  - BerkMin uses learned information to increase emphasis on “locality” of decisions
2. With conflict analysis, decision can restart any time
  - Change to different decision ordering heuristic to explore different areas in the input space
3. Modify the learned information
  - Remove least-used learned information
  - Simplify or synthesize the learned information
  - Any other idea?

# What affect the SAT efficiency?

1. Decision order
2. Logic implication (Boolean Constraint Propagation, BCP)
3. Various learning techniques
- ▶ 4. Database simplification

# Simplify SAT Database. Why bother?

1. CNF proof instances generated from real-life problems (e.g. assertions in a circuit) are usually quite redundant
  - Better clausifier?
2. During SAT proof, the number of added learnt clauses will become much larger than the number of original clauses
  - A few thousands vs. millions
3. Slimmer clause database usually implies better proof efficiency

# Many SAT proof database simplification techniques...

- ◆ Especially for CNF...
  1. Variable Elimination by Clause Distribution
  2. Clause Subsumption
  3. Self-Subsuming Resolution
  4. Simplification by Definition of a Gate
  5. Blocked Clause Elimination
  6. Equivalent Literal, Pure Literal Elimination, etc
- ◆ Also, many techniques to generate “better” CNF instances (from circuit problems)
  1. Tseitin Transformation
  2. Plaisted-Greenbaum Encoding
  3. Utilization of Logic Synthesis Techniques
- ◆ Note: in the following slides, ' for a literal/variable means negation; for clause/problem means another one.

# Satisfiability Equivalent Problem

- ◆ A SAT proof instance P1 is a “satisfiability equivalent (SAT-EQ) problem” of another proof instance P2 iff:
  - P1 is SAT implies P2 is SAT, and
  - P1 is UNSAT implies P2 is UNSAT

→ Note that P1 is NOT necessarily logically equivalent to P2
- ◆ “Resolution” preserves the SAT-EQ
  - Let  $\otimes$  be the resolution operator,  
clauses  $c_1 = (x + a_1 + \dots + a_n)$ ,  $c_2 = (x' + b_1 + \dots + b_m)$   
and  $c = c_1 \otimes c_2$  is the resolvent of  $c_1$  and  $c_2$
  - $c = (a_1 + \dots + a_n + b_1 + \dots + b_m)$
  - $(c_1 \wedge c_2)$  implies  $c$
  - $(c_1 \wedge c_2)$  and  $c$  are SAT-EQ

# Variable Elimination by Clause Distribution

- ◆ In a CNF proof instance  $S$ , let  $S_x$  and  $S_{x'}$  be the sets of clauses in which  $x$  and  $x'$  occurs, respectively.
  - $S = S_x \cup S_{x'}$
- ◆ Resolution operation can be lifted to sets of clauses as:
  - $S' = S1 \otimes S2 = \{ C1 \otimes C2 \mid \forall C1 \in S1, \forall C2 \in S2 \}$ 
    - $x$  will be eliminated from  $S$
    - $S'$  is SAT-EQ to  $S$
    - Is  $S'$  always simpler than  $S$ ?
    - $S'$  may contain several trivial clauses. (A clause is called trivial if it contains a variable and its negation)
    - $S'$  also contains many subsumed clauses

# Clause Subsumption

- ◆ A clause  $C1$  is said to (syntactically) subsume another clause  $C2$  if  $C1 \subseteq C2$ 
  - e.g.  $(a + b)$  subsumes  $(a + b + c)$
  - A subsumed clause is redundant in a SAT problem and can be removed from the proof
- ◆ [Remember] Variable eliminations by clause distribution usually lead to many subsumed clauses
- ◆ How to identify the subsumed clauses in a CNF?
  - [Ref: Een SAT2005]
    1. For each clause, a 64-bit signature is stored.
    2. Each literal is hashed to  $0 \dots 63$ .
    3. The signature = bitwise\_OR of the hashed literal indices
    4. Occur\_list: literal  $\rightarrow$  clauses
    5. Check subsumptions with the aid of the clause signatures (avoid useless checking)

# Self-Subsuming Resolution

- ◆ It's often that one clause can “almost subsume” the other. For example:
  - $C1: (x' + a), C2: (x + a + b)$ 
    - $C1$  does not subsume  $C2$
  - But if we do  $C2' = C2 \otimes C1 = (a + b)$ 
    - $C2'$  will subsume  $C2$
    - We say “ $C2$  is strengthened by self-subsumption using  $C1$ ”  
(i.e. Problem becomes:  $(x' + a) (a + b)$  )
- ◆ Self-subsumption is a powerful technique in simplifying CNF database



# Simplification by Definition of a Gate

- ◆ There are usually many functionally dependent variables in a CNF. For example:
  - ...  $(x + a' + b')(x' + a)(x' + b)$  ...
  - $x$  is actually equal to " $a \wedge b$ "
  - We call the equation " $x = a \wedge b$ " the definition of  $x$
  - Can we remove the variable  $x$ ?
- ◆ [Fact] If  $x$  has a definition and is eliminated by clause distribution, many redundant resolvents are generated.
  - [e.g.]      1          2          3          4          5          6  
                $(x + c)(x + d')(x + a' + b') (x' + a)(x' + b)(x' + e' + f)$
  - The resolvents are:
    - 1⊗4    1⊗5      2⊗4    2⊗5          3⊗6  
 $(c + a) (c + b) (d' + a) (d' + b) (a' + b' + e' + f) \rightarrow A$
    - 3⊗4              3⊗5  
 $(a' + b' + a) (a' + b' + b) \rightarrow B$
    - 1⊗6              2⊗6  
 $(c + e' + f) (d' + e' + f) \rightarrow C$
  - We will show in the next slide that  $A$  implies  $B \cup C$

# Simplification by Definition of a Gate

- ◆ Let a CNF  $S$  contains a definition of a variable  $x$ , that is,  $x = a \wedge b$  ---
    - ...  $(x + a' + b')(x' + a)(x' + b)$  ...
  - ◆ Let  $S = G \cup R$ ,  
 $G = (x + a' + b')(x' + a)(x' + b)$   
 $R = S \setminus G$
  - ◆ Let  $G_x$  and  $G_{x'}$  ( $R_x$  and  $R_{x'}$ ) be the set of clauses of  $G$  ( $R$ ) in which  $x$  and  $x'$  occurs, respectively.
    - $S = (G_x \cup R_x) \cup (G_{x'} \cup R_{x'})$
    - $S' = (G_x \cup R_x) \cup (G_{x'} \cup R_{x'}) = S'' \cup G' \cup R'$   
where ---  
 $S'' = (R_x \otimes G_{x'}) \cup (G_x \otimes R_{x'})$   
 $G' = G_x \otimes G_{x'}$   
 $R' = R_x \otimes R_{x'}$
- $S''$  implies  $G' \cup R'$   
That is,  $G'$  and  $R'$  are redundant and thus can be removed

# Simplification by Definition of a Gate

- ◆ In the previous example ---

- Example:

$$(x + c)(x + d')(x + a' + b') (x' + a)(x' + b)(x' + e' + f)$$

Becomes...

$$\begin{aligned} & ( (x + a' + b') \otimes (x' + e' + f) ) \\ & ( \{ (x + c), (x + d') \} \otimes \{ (x' + a), (x' + b) \} ) \end{aligned}$$

$$\rightarrow (a' + b' + e' + f) (c + a) (c + b) (d' + a) (d' + b)$$

# Blocked Clause Elimination (ref: Järvisalo TACAS 10)

- ◆ Blocking literal
  - A literal  $l$  in a clause  $C$  of a CNF  $F$  blocks  $C$  (w.r.t.  $F$ ) if for every clause  $C' \in F$  with  $l' \in C'$ , the resolvent of  $(C \otimes C')$  on  $l$  is a tautology.
- ◆ Blocked clause
  - A clause is blocked if it has a literal that blocks it
  - Removal of a blocked clause reserves satisfiability
- ◆ Example:  $(a' + b) (a + b' + c') (a' + c)$ 
  - $c$  blocks  $(a' + c)$  because  $(a + b' + c') \otimes_c (a' + c) = 1$
  - Problem becomes  $(a' + b) (a + b' + c')$
  - $b'$  blocks  $(a + b' + c') \rightarrow$  Problem becomes  $(a' + b)$
  - Problem is satisfiable!!

# Equivalent Literal, Pure Literal Eliminations

## ◆ Equivalent Literal

- If both  $(a + b')$  and  $(a' + b)$  exist in CNF,  $a$  and  $b$  are equivalent  $\rightarrow$  Replace  $b$  with  $a$
- If  $(a + b')$ ,  $(b + c')$  and  $(c + a')$  exist in CNF,  $a$ ,  $b$  and  $c$  are equivalent  $\rightarrow$  Pick one representative literal

## ◆ Pure Literal Eliminations

- If some variable exists only in one phase in all the clauses it appears (i.e. pure literal) ---
  - $\rightarrow$  Assigning these literals to '1' preserves satisfiability
  - $\rightarrow$  Removal of these clauses preserves satisfiability

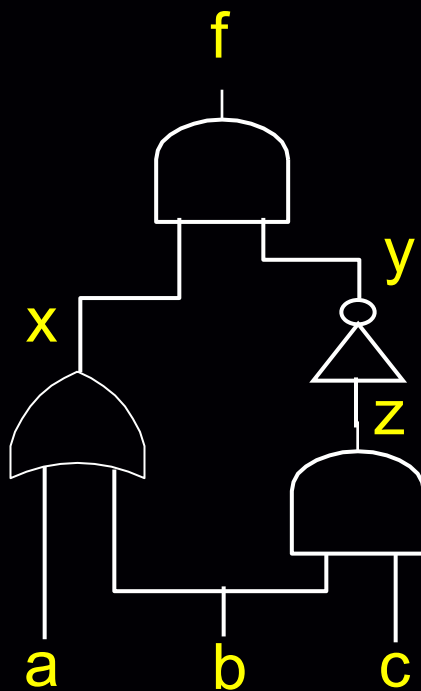
# Transforming Circuit Problems for CNF SAT

- ◆ Although the CNF and circuit-based SAT can be equally efficient, however, there are much more existing CNF solvers than circuit SAT.
  - It's often a need to transform a circuit problem to CNF
- 1. Tseitin Transformation
- 2. Plaisted-Greenbaum Encoding
- 3. Utilization of Logic Synthesis Techniques

# Tseitin Transformation

1. Assign each gate with a variable
2. For each gate, generate CNF clauses for its input and output variables

◆ Example:

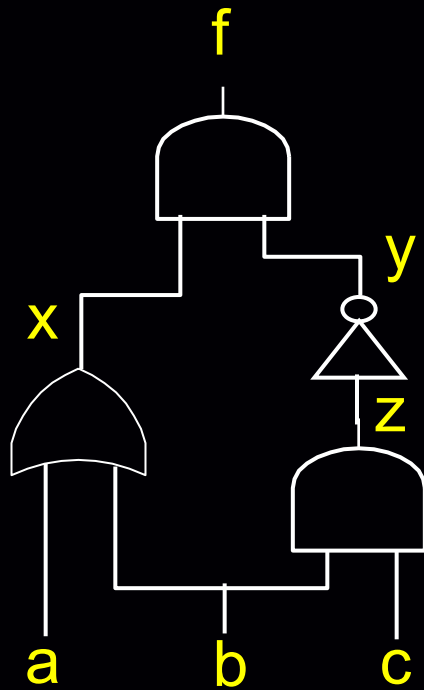


$$\begin{aligned} F: & f (f \longleftrightarrow x \wedge y) (x \longleftrightarrow a \vee b) (y \longleftrightarrow z') (z \longleftrightarrow b \wedge c) \\ \equiv & f (f \rightarrow x) (f \rightarrow y) (f' \rightarrow x' \vee y') \\ & (x' \rightarrow a') (x' \rightarrow b') (x \rightarrow a \vee b) \\ & (y \rightarrow z') (y' \rightarrow z) \dots \\ \equiv & f (f' + x) (f' + y) (f + x' + y') \\ & (x + a') (x + b') (x' + a + b) \\ & (y' + z') (y + z) \dots \end{aligned}$$

**However, many redundant variables/  
clauses are generated...**

# Plaisted-Greenbaum Encoding

- Starting from the target  $f$ , traverse the circuit in topological order
  - For current visited variable  $g$ , if it is unate, remove the clause(s) that is(are) trivially implied (i.e.  $\text{false} \rightarrow P$ ). If it is binate, continue as Tseitin transformation.



$$\begin{aligned}
 F: & f (f \longleftrightarrow x \wedge y) (x \longleftrightarrow a \vee b) (y \longleftrightarrow z') (z \longleftrightarrow b \wedge c) \\
 \equiv & f (f \rightarrow x \wedge y) (x \longleftrightarrow a \vee b) (y \longleftrightarrow z') (z \longleftrightarrow b \wedge c) \\
 \equiv & f (f' + x) (f' + y) (x \rightarrow a \vee b) (y \longleftrightarrow z') (z \longleftrightarrow b \wedge c) \\
 \equiv & f (f' + x) (f' + y) (x' + a + b) (y \rightarrow z') (z \longleftrightarrow b \wedge c) \\
 \equiv & f (f' + x) (f' + y) (x' + a + b) (y \rightarrow b' \vee c') \\
 \equiv & f (f' + x) (f' + y) (x' + a + b) (y' + b' + c')
 \end{aligned}$$



# Utilization of Logic Synthesis Techniques

- ◆ How many are there n-input Boolean functions?
  - e.g. 65536 for 4-input functions
  - Are they all distinct?
- ◆ NPN-equivalent functions
  - Two functions are called NPN-equivalent iff they are equivalent by negating parts of the inputs and outputs, and by permuting inputs
  - e.g.  $(a \wedge b)$  and  $(b \vee a')$  are NPN-EQ
- ◆ How many are there distinct NPN-EQ n-input Boolean functions?
  - Well, no general formula...
  - 1-input: 2; 2-input: 4; 3-input: 14; 4-input: 222; 5-input: ??

$$2^{2^n}$$

# Utilization of Logic Synthesis Techniques

- ◆ How to utilize this NPN-EQ concept in generating CNF formula from circuit?
    - For each NPN-EQ class, derive the “best” CNF representation
      - CNF representations of NPN functions are different only on literal phases (push the output negation forward)
    - Partition the circuit into clusters of n-input “macro cells”. That is, each macro cell has exactly n inputs and 1 output.
      - e.g. 4-input
    - For each macro cell, generate the best CNF based on its NPN class
      - How do we know which NPN class does this macro cell belongs to? (Hint: 16-bit parallel pattern simulation + table lookup)
- => (Hint) Parallel pattern simulations for 65536 patterns

# What we have learned...

- ◆ What is the Boolean Satisfiability (SAT) problem?
- ◆ Circuit SAT vs. CNF SAT
- ◆ Key factors for SAT efficiency
  - Boolean constraint propagation (BCP)
  - Decision ordering
  - Various learning
  - Database simplification